

Module : Cerner les besoins matériels dans le contexte de l'IA

CORRIGÉ — PARTIE 2

Travaux Pratiques

Rappel du scénario

Startup en reconnaissance d'images médicales. Entraînement sur 500 000 images (224×224 px, 3 canaux). Déploiement via API REST. Budget : 25 000 \$.

Exercice 1 — Conception de la chaîne CI/CD (30 pts)

1.1 — Tableau des composantes CI/CD (15 pts)

Réponse modèle :

Composante	Outil choisi	Justification
Gestionnaire de code source	Git + GitHub	Git enregistre l'historique complet du code. GitHub héberge le dépôt dans le cloud, permet la collaboration (pull requests, code review) et intègre GitHub Actions pour la CI/CD. Gratuit pour les dépôts privés.
Serveur CI	GitHub Actions	Intégré nativement à GitHub, sans serveur à gérer. Déclenche automatiquement les tests à chaque push. Alternatives acceptées : Jenkins (auto-hébergé, plus de contrôle), GitLab CI (si GitLab choisi).
Gestionnaire de données	DVC	DVC (Data Version Control) versionne les datasets comme Git versionne le code. Crée des fichiers .dvc légers dans Git qui pointent vers les données stockées sur S3/GCS/NAS. Indispensable en IA : même code + données différentes = résultats différents.
Tracking d'expériences	MLflow	Enregistre automatiquement chaque run d'entraînement : hyperparamètres, métriques (accuracy, loss, F1), artefacts (modèle .pt/.h5). Interface web pour comparer les expériences. Alternatives : Weights & Biases (W&B), Neptune.
Registre de modèles	MLflow Model Registry	Stocke les modèles versionnés avec leurs métriques. Gère les transitions staging → production. Permet de rollback vers une version précédente si la nouvelle est moins performante.

Barème 1.1 : 3 pts par composante (1 pt outil correct, 2 pts justification pertinente).
 Accepter Docker/Kubernetes comme composante supplémentaire (bonus). Pénaliser si l'étudiant confond les rôles (ex : MLflow pour le versionnage de code).

1.2 — Flux CI/CD automatisé (15 pts)

Réponse modèle — 5 étapes :

Étape 1 — Push & Déclenchement

Le développeur pousse (push) son code sur la branche main (ou via une pull request). GitHub Actions détecte le push et déclenche automatiquement le pipeline CI/CD.

Étape 2 — Tests automatiques

Le serveur CI exécute les tests unitaires (vérification du code Python), les tests d'intégration (vérification du pipeline de données), et les linters (qualité du code). Si un test échoue → le pipeline s'arrête et le développeur est notifié.

Étape 3 — Entraînement du modèle

Si les tests passent, le pipeline lance l'entraînement du modèle sur le serveur GPU. DVC récupère la version correcte du dataset. Les hyperparamètres sont lus depuis un fichier de configuration versionné (params.yaml).

Étape 4 — Évaluation & Tracking

MLflow enregistre les métriques (accuracy, F1-score, loss) et compare avec le modèle actuellement en production. Si le nouveau modèle est meilleur → il est promu dans le Model Registry. Sinon → alerte et arrêt.

Étape 5 — Déploiement automatique

Le modèle validé est empaqueté dans un conteneur Docker et déployé sur l'API REST (via Kubernetes ou un serveur Flask/FastAPI). Le monitoring (Prometheus, Grafana) vérifie que l'API répond correctement. Rollback automatique si erreur.

Barème 1.2 : 3 pts par étape (1 pt pour l'idée, 2 pts pour le détail). Accepter des variantes (ex : étape de validation manuelle avant déploiement). L'étudiant qui mentionne le rollback automatique : bonus +2 pts.

Exercice 2 — Sélection du matériel IA (40 pts)

2.1 — Configuration matérielle recommandée (20 pts)

Réponse modèle :

Composante	Modèle / Spec	Qté / Cap.	Justification IA
GPU	NVIDIA RTX 4090	2×	24 Go VRAM chacun. Pour 500K images 224×224 : un modèle ResNet-50 tient dans 8 Go, mais les batches de 64-128 images et les gradients nécessitent ~16-20 Go. 2 GPU permettent l'entraînement distribué (DataParallel). Prix : ~1 600 \$/unité.
CPU	AMD Ryzen 9 7950X (16 cœurs)	1×	16 cœurs / 32 threads. Ratio 8 cœurs/GPU respecté. Charge les données en parallèle (DataLoader workers). Le CPU prépare les batches pendant que le GPU calcule → pas de goulot. Prix : ~550 \$.
RAM	DDR5 4800 MHz	128 Go	500K images × 224×224×3 = ~70 Go en mémoire si chargées intégralement. 128 Go laisse de la marge pour l'OS, les pré-traitements et les buffers. Règle : RAM ≥ 2× VRAM totale. Prix : ~300 \$.
Stockage NVMe	Samsung 990 Pro Gen4	2 To	Dataset brut ~35 Go + augmentations + checkpoints modèles + OS. NVMe Gen4 : 7 Go/s lecture → les batches sont chargés aussi vite que le GPU les consomme. Évite le bottleneck I/O. Prix : ~150 \$.
Stockage HDD	Seagate IronWolf	8 To	Archivage des datasets historiques, backups, logs d'expériences. Pas besoin de vitesse pour le stockage froid. Prix : ~180 \$.
PSU	Corsair RM1200x (80+ Gold)	1200W	2× RTX 4090 = 2×450W = 900W + CPU 170W + reste ~100W = ~1170W. PSU 1200W avec certification 80+ Gold pour l'efficacité. Marge minimale mais suffisante. Prix : ~200 \$.
Réseau	Carte Ethernet 10 GbE	1×	Transfert rapide des datasets depuis le NAS/serveur de données. 10 Gbps = 1.25 Go/s. Pour un dataset de 35 Go : ~28 secondes. En 1 Gbps : ~5 minutes. Prix : ~80 \$.

Barème 2.1 : ~3 pts par composante. 1 pt pour un choix raisonnable, 2 pts pour la justification IA (pas juste 'c'est rapide' — il faut expliquer POURQUOI c'est nécessaire pour CE projet). Accepter RTX 3090 ou A5000 comme alternatives GPU. Accepter Intel Xeon pour le CPU.

2.2 — Calcul du budget (10 pts)

Réponse modèle :

Composante	Prix unitaire	Total
2× NVIDIA RTX 4090	1 600 \$	3 200 \$
AMD Ryzen 9 7950X	550 \$	550 \$
Carte mère (ex: ASUS ProArt X670E)	450 \$	450 \$
128 Go DDR5 (4×32 Go)	300 \$	300 \$
SSD NVMe 2 To Gen4	150 \$	150 \$
HDD 8 To	180 \$	180 \$
PSU 1200W 80+ Gold	200 \$	200 \$
Boîtier tour (ventilé)	150 \$	150 \$
Carte réseau 10 GbE	80 \$	80 \$
Refroidissement (ventilateurs)	100 \$	100 \$
TOTAL		~5 360 \$

Analyse budget : Le budget de 25 000 \$ est LARGEMENT respecté (~5 400 \$ soit ~22%). Le reste du budget peut être alloué à : (1) licences logicielles ou abonnement cloud pour les pics de charge, (2) un deuxième serveur pour l'inférence/API, (3) du stockage réseau (NAS), (4) des salaires développeurs. L'étudiant qui propose une allocation du budget restant : bonus.

Barème 2.2 : 5 pts pour un calcul correct et détaillé. 3 pts pour l'analyse (budget respecté ou dépassé + justification). 2 pts pour les compromis ou l'allocation du restant. Accepter des prix différents si cohérents avec le marché actuel.

2.3 — Espérance de vie et remplacement (10 pts)

Réponse modèle :

Composante	Durée de vie IA	Recommandation de remplacement
GPU	2-3 ans	La VRAM est le facteur limitant. Les modèles grossissent chaque année. Une RTX 4090 (24 Go) sera insuffisante pour les LLM de 2027. Remplacer dès qu'un nouveau modèle nécessite plus de VRAM que disponible.
CPU	4-5 ans	Le CPU évolue moins vite que le GPU en IA. Un 16 cœurs restera pertinent tant que le ratio CPU/GPU est respecté. Remplacer uniquement si on ajoute des GPU.
RAM	5-7 ans	La RAM DDR5 restera compatible longtemps. Augmenter la capacité (256 Go) avant de remplacer. Remplacer si passage à DDR6.
SSD NVMe	3-5 ans	L'usure dépend du volume d'écriture (TBW). Un entraînement IA écrit beaucoup de checkpoints. Surveiller via S.M.A.R.T. Remplacer quand les performances dégradent ou que l'espace est insuffisant.
HDD	3-5 ans	Pièces mécaniques → usure. En datacenter, taux de panne augmente après 3 ans. Prévoir des backups RAID ou cloud.
PSU	7-10 ans	Composante la plus durable si bien dimensionnée. Remplacer si ajout de GPU qui dépasse la capacité ou si des instabilités apparaissent.

Barème 2.3 : ~1.5 pts par composante. 0.5 pt pour une durée réaliste, 1 pt pour la recommandation contextualisée IA. L'étudiant qui mentionne le monitoring S.M.A.R.T. ou le TBW des SSD : bonus.

Exercice 3 — Analyse et optimisation (30 pts)

3.1 — Analyse PCIe pour 4× A100 (15 pts)

Réponse modèle :

CPU recommandé : AMD EPYC 7543 (32 cœurs) ou Intel Xeon w9-3495X (56 cœurs). Le ratio de 4-8 cœurs/GPU donne : $4 \text{ GPU} \times 8 = 32$ cœurs minimum.

Lignes PCIe nécessaires : Chaque A100 nécessite 16 lignes PCIe Gen4 ($\times 16$). Pour 4 GPU : $4 \times 16 = 64$ lignes PCIe minimum. Un AMD EPYC offre 128 lignes PCIe Gen4 → largement suffisant. Un Intel Xeon W offre 112 lignes → suffisant. Un Ryzen grand public n'offre que 28 lignes → INSUFFISANT pour 4 GPU en $\times 16$.

Calcul de bande passante : PCIe Gen4 $\times 16 = \sim 32$ Go/s par GPU. Si les 4 GPU tournent en mode $\times 8$ (faute de lignes) : bande passante divisée par 2 → goulot d'étranglement lors des transferts CPU↔GPU et lors de l'entraînement distribué multi-GPU.

NVLink : Pour 4× A100, NVIDIA recommande NVLink pour la communication directe GPU↔GPU (600 Go/s pour NVLink 4.0 vs 32 Go/s pour PCIe). Indispensable pour l'entraînement distribué efficace de grands modèles. Nécessite une carte mère compatible (NVIDIA HGX ou SXM).

Barème 3.1 : CPU correct avec justification cœurs (4 pts). Calcul lignes PCIe correct (4 pts). Explication du goulot si lignes insuffisantes (4 pts). Mention NVLink (3 pts bonus). Pénaliser si l'étudiant propose un CPU grand public (Ryzen, Core i9) pour 4× A100.

3.2 — GPU on-premise vs TPU Cloud (15 pts)

Réponse modèle :

Critère	GPU on-premise	TPU Google Cloud
Coût initial	Élevé ($\sim 5\,000$ - $50\,000$ \$ selon config). Investissement unique mais immobilisation de capital.	Quasi nul. Paiement à l'usage (TPU v4 : ~ 3.22 \$/heure). Pas d'achat matériel.
Coût long terme	Faible après achat. Coûts récurrents : électricité (~ 200 - 500 \$/mois pour 4 GPU), refroidissement, maintenance. Amorti en 12-18 mois si utilisation intensive.	Peut devenir très élevé si utilisation continue. $4 \times \text{TPU v4 } 24\text{h}/7\text{j} = \sim 9\,200$ \$/mois. Avantageux uniquement pour des pics ponctuels.

Performances	A100/H100 : excellent pour tout type de modèle. Flexibilité totale (PyTorch, TensorFlow, modèles custom). Latence réseau nulle.	TPU v4 : 275 TFLOPS, optimisé pour TensorFlow et JAX. Moins flexible avec PyTorch (support expérimental). Latence réseau variable.
Flexibilité	Totale. Choix de l'OS, des frameworks, des bibliothèques. Accès physique au matériel. Possible de modifier le hardware.	Limitée à l'écosystème GCP. Dépendance au fournisseur cloud (vendor lock-in). Pas de personnalisation hardware.
Maintenance	À la charge de l'entreprise. Remplacement de composants, mises à jour drivers, monitoring, refroidissement. Nécessite des compétences sysadmin.	Aucune. Google gère tout : matériel, refroidissement, mises à jour. L'équipe se concentre uniquement sur le code IA.

Recommandation pour cette startup : Pour ce scénario précis, le GPU on-premise est recommandé. Raisons : (1) Le budget de 25 000 \$ permet une config solide (2× RTX 4090) pour ~5 400 \$. (2) L'entraînement sur 500K images est récurrent (itérations, expériences) → le coût cloud deviendrait prohibitif en quelques mois. (3) L'imagerie médicale impose souvent des contraintes de confidentialité (données patient) → garder les données on-premise est plus sûr réglementairement. (4) Le restant du budget (~20 000 \$) peut financer un petit quota cloud pour les pics de charge exceptionnels (approche hybride).

Cas où le TPU Cloud serait préférable : Si la startup n'avait PAS de budget matériel, ou si elle devait scaler rapidement à 64+ GPU pour un très grand modèle, ou si elle utilisait exclusivement TensorFlow/JAX.

Barème 3.2 : Tableau comparatif correct (8 pts — ~1.5 pt par critère). Recommandation argumentée (5 pts). L'étudiant qui mentionne la confidentialité des données médicales : bonus +2 pts. L'étudiant qui propose une approche hybride (on-premise + cloud pour les pics) : bonus +2 pts.

Notes générales pour la correction

Tolérance sur les prix : Les prix varient selon la date et le marché. Accepter tout prix réaliste ($\pm 30\%$). L'important est la cohérence du calcul et la justification, pas le prix exact.

Alternatives acceptables : GPU : RTX 3090, A5000, A6000 (si justifié). CPU : Intel Xeon, AMD Threadripper. Serveur CI : Jenkins, GitLab CI, CircleCI. Tracking : W&B, Neptune. Ces alternatives sont tout aussi valides si correctement justifiées.

Valoriser la réflexion : Un étudiant qui fait un choix différent du corrigé mais qui le justifie bien (ex : 1 seul GPU A6000 48 Go au lieu de 2× RTX 4090) doit être noté autant que le corrigé. L'essentiel est la compréhension des contraintes IA, pas la mémorisation d'un modèle précis.

Bonus possibles (5 pts max) : Mention du refroidissement, calcul du TDP total, mention NVLink, approche hybride cloud, contraintes réglementaires données médicales, mention de la mixed precision (float16) pour économiser la VRAM.